
Toward Engineering Rigor in AI-Assisted Software Development

A Cooperative Audit Architecture for Intent-Indexed,
Provenance-Aware Development Workflows

Justin Cheshire

Field Operations Manager, Fuse Solutions & Independent AI Systems Developer

justincheshire.com · [github: solring](https://github.com/solring)

March 2026

Abstract

Software development has long resisted the structural rigor that characterizes mature engineering disciplines. Unlike civil, aerospace, or pharmaceutical engineering — where intent, decision, and artifact are bound together in traceable records — software production has historically treated the reasoning that produces code as invisible and ephemeral. The emergence of large language model (LLM) assisted development accelerates this problem: AI-generated code compounds at velocity that outpaces human review, while the intent behind that code remains unrecorded. This paper proposes a cooperative audit architecture — the Gothi/Gethli system — as a structural response to the engineering rigor gap. By treating working AI agents as cooperative participants in a declarative audit framework rather than passive subjects of external monitoring, the system captures intent-indexed event records at the moment of generation, enabling provenance-aware, drift-detectable, and replayable audit trails without continuous code ingestion. The architecture draws on event sourcing principles, safety-critical traceability standards, and the epistemological distinction between audit and validation to define a new primitive: a cooperative epistemic layer for AI-assisted software systems.

§ 1

The Engineering Rigor Gap in Software Development

Software engineering occupies an unusual position among technical disciplines. It claims the word "engineering" while operating largely without the structural accountability mechanisms that define engineering practice in other domains. A civil engineer cannot submit a bridge design without a traceable calculation record linking every structural claim to its underlying load assumption. An aerospace software system flying at Development Assurance Level A under DO-178C must demonstrate end-to-end, bidirectional traceability from system requirements through design, code, and verification artifacts.¹ A pharmaceutical process is governed by Good Laboratory Practice, which mandates contemporaneous documentation of every procedural step and decision. In each of these disciplines, the reasoning that produced the artifact is itself part of the deliverable.

Software development has historically lacked this property. The gap between what was intended and what was built has been largely invisible until something failed in production. Code reviews, test suites, and documentation practices help but remain fundamentally retrospective — they examine artifacts after the fact and depend on practitioners remembering to look. As the empirical software engineering research community has noted for decades, developer beliefs about their own software frequently diverge from what the evidence actually shows.² The reasoning layer — the sequence of decisions, approach switches, and intent boundaries that produced the code — has been structurally absent from the

development record.

Traceability research in software engineering has addressed part of this problem. Requirements traceability — the ability to follow a requirement forward through design, implementation, and verification — has been extensively studied.³ In practice, however, traceability is "conducted in an ad-hoc, after-the-fact manner" and its benefits are frequently unrealized as a result.⁴ Even in organizations that invest in traceability tooling, the links tend to document artifact relationships rather than the reasoning that created those relationships. The decision topology of a session — why an approach was chosen, when it was abandoned, what prior outputs were superseded — remains outside the traceability record.

Provenance-aware computing research has named this gap precisely.⁵ Much software, as one analysis puts it, "exists for which there is no provenance-awareness, i.e. at best, the data it outputs can be connected to its inputs, but with no record of intermediate processing." The intermediate processing — the reasoning — is exactly what has been missing.

§ 2

AI-Assisted Development and the Accelerating Provenance Problem

The emergence of large language model-assisted software development has not introduced the provenance gap — it has dramatically widened it. As of 2024–2025, the majority of professional developers report daily use of AI coding assistants, with some organizations reporting that AI-generated code accounts for a quarter or more of their total output.⁶ The velocity gain is real: developer surveys report productivity increases as the primary benefit, with 82.7% of respondents citing accelerated output.⁷ The accountability structure has not kept pace.

The quality picture is complicated. GitClear's analysis of 211 million changed lines of code across major repositories found a measurable increase in code churn and duplicate code blocks coinciding with the rise of AI assistance.⁸ Qodo's 2025 State of AI Code Quality report found that 65% of developers using AI for refactoring cite "missing context" as the primary cause of poor outputs — more frequently than hallucination — and that only 3.8% of developers report both low hallucination rates and high confidence in shipping AI code.⁹ The MIT Technology Review summarized the fundamental structural problem: LLMs "can hold only a limited amount of information in their context window" and "if you tell it to do a dozen things, it'll do 11 of them and just forget that last one."¹⁰

The problem is not merely that AI-generated code may be incorrect. The deeper problem is that when AI generates code at high velocity across long sessions, the reasoning that produced that code — the approach selections, the approach switches, the superseded designs — is neither recorded nor accessible for review. A developer reviewing AI-assisted output sees the artifact; they do not see the decision record

that produced it. This asymmetry between velocity and accountability is the defining challenge of AI-assisted software engineering.

Experienced practitioners have responded by developing more structured workflows. Addy Osmani, an engineering leader at Google, describes the emerging best practice as treating LLMs as "a powerful pair programmer that requires clear direction, context and oversight rather than autonomous judgment," with explicit specification documents and architecture decision records.¹¹ This represents a practitioner-level attempt to impose engineering discipline through cultural practice. The approach described in this paper makes that discipline structural rather than optional — embedded in the tooling rather than dependent on individual rigor.

§ 3

The Audit vs. Validation Distinction

A foundational design decision of the proposed architecture is the explicit rejection of "validation" as its governing concept in favor of "audit." This distinction is not cosmetic — it resolves an epistemological problem that would otherwise undermine the entire system.

Validation implies a standard being checked against. A validation system must encode what "correct" means in the domain it monitors, which presupposes either formal specifications precise enough to serve as a reference (rare in general software development) or a judgment capacity that can evaluate semantic correctness (beyond the reach of current automated systems in most domains). The American Scientist's survey of empirical software engineering research notes that software development has historically struggled even to agree on what quality means, let alone to measure it reliably — the discipline is "at a turning point comparable to the dawn of evidence-based medicine."¹² A system that claims to validate AI output is making a promise it cannot keep in any domain where correctness is contested.

Audit, by contrast, implies a record being examined. An audit does not certify truth — it certifies traceability, consistency, and the completeness of the record. This is precisely what safety-critical engineering standards require. DO-178C, the aviation software certification standard, does not require that code be proven correct in any absolute sense. It requires that every software requirement be traceable to a design artifact, every design artifact be traceable to code, and every claim be traceable to evidence.¹³ The standard is an audit framework, not a correctness proof. NASA's Software Engineering Handbook similarly defines traceability as the ability to "follow the life of a requirement in both a forwards and backwards direction" — a provenance requirement, not a validation requirement.¹⁴

The cooperative audit architecture proposed here adopts the same epistemological posture: it claims to know what changed, where it came from, what contradicts what, and how reliably the agents participating in its audit are doing their part. It makes no claim about whether any of this is correct. That honest scope is the architecture's most important feature — because it is a scope the system can actually fulfill, indefinitely, at any scale, in any domain.

§ 4

Event Sourcing as a Cognitive Architecture Primitive

The cooperative audit architecture draws on a well-established pattern from distributed systems engineering: event sourcing. Greg Young, who formalized the concept and coined the term CQRS (Command Query Responsibility Segregation), describes the critical distinction between a transaction log and an event log: an event log, unlike a transaction log, captures intent. "Consider the difference between 'INSERT: customer record' and 'CustomerAccountOpened': the former records a state mutation; the latter records a business decision."¹⁵ Event-sourced systems are append-only and immutable — they preserve the history of what happened and why, not merely the current state.

The RoboCouncil review of the Gothi/Gethli architecture identified this connection precisely: the system is "basically event sourcing for cognition" — intent-indexed event sourcing with post-hoc semantic auditing. This framing is significant because it connects a novel AI orchestration problem to a mature engineering pattern with well-understood properties: deterministic capture, deferred compute, re-runnable audits, and an immutable append-only log that cannot be revised to fit a preferred narrative.

The extension of event sourcing to the cognitive layer introduces a new requirement: events must be contributed by the agents producing the work, not reconstructed afterward from artifacts. In a traditional event-sourced financial system, events are generated by the system's own state transitions — a completed trade, a balance update, an order cancellation. In a cooperative audit system for AI development, the events that matter most are intent boundaries: a working agent switching approach, superseding a prior recommendation, flagging an ungrounded claim. These events are invisible to any monitoring system that can only observe outputs. They require the agent itself to participate in their creation.

This is the cooperative capture principle: working agents are not passive subjects of external monitoring, but cooperative participants in the audit of their own outputs — without being given authority over the interpretation of that participation. The distinction is structural and essential. The agent flags; the audit reasoner classifies. Self-certification is architecturally prevented.

§ 5

The Gothi/Gethli Architecture

5.1 The Three-Party Model

The cooperative audit architecture separates concerns across three parties with strictly non-overlapping responsibilities. The working agent — any AI system generating code, analysis, or recommendations — emits structured event markers at audit-relevant moments during generation. Gothi, the ingestion controller, captures these markers alongside reactive signals (terminal errors, output hash drift) and writes raw event records to a persistent audit database. Gethli, the audit reasoner, operates in a second pass over those records — classifying events, detecting patterns, and producing assessed findings without any access to the generation pipeline it audits.

The two-pass architecture is load-bearing. Gothi writes raw records immediately, without waiting for Gethli's analysis. Gethli runs on demand or at checkpoints, against the accumulated record. This separation means capture is never blocked by analysis, analysis is always working against a complete and unmodified record, and the same record can be re-analyzed as Gethli's capabilities improve. The audit database is immutable by design: the raw agent flag field is never overwritten after write.

5.2 The Declarative Trigger Registry

The event flag system is built on a declarative trigger registry derived from feature flag architecture. Each audit trigger is a named, typed, resolvable condition: a key, a category, a decide function, a capture scope, and a schema version. The registry is append-only — new triggers are additive and never modify existing trigger behavior. Every audit record carries the trigger registry version, flag schema version, and agent profile version in effect at the time of emission, ensuring records are interpretable regardless of when they were generated.

The core trigger set covers five cooperative triggers and three reactive triggers. Cooperative triggers — `APPROACH_SWITCH`, `OUTPUT_SUPERSEDES`, `GROUNDING_GAP`, `CONTRADICTION`, and `MODEL_UNCERTAINTY` — require agent participation to fire. Reactive triggers — `terminal-error`, `drift-detected`, and `session-checkpoint` — fire independently of agent behavior, forming the fallback layer that ensures the audit trail remains meaningful even when cooperative participation degrades.

5.3 The Cooperative Capture Layer

Cooperative capture is implemented through an audit contract embedded in the working agent's profile as a mandatory, gate-style instruction executed before every response. The framing is deliberately structural rather than aspirational — not "consider flagging when appropriate" but a blocking requirement with explicit trigger conditions, strict grammar, rate controls, and a clear statement of what the agent is responsible for (surfacing claims) and what it is not responsible for (classifying findings).

The inline marker grammar requires a specific format: `[[AUDIT:TRIGGER_KEY | context: description]]`, with an optional reference field for triggers that supersede prior artifacts. Gothi's flag parser validates every marker against the schema, records parse status on every record, and enforces rate limiting and deduplication — maximum three flags per response, thirty-second deduplication window —

to prevent noise flooding. Malformed markers do not crash the capture pipeline; they are logged with `flag_parse_status: invalid` and treated as audit data about the agent's compliance behavior.

5.4 The Audit Event Record

Every event written to the audit database — whether from cooperative agent flags or reactive capture — uses a canonical record shape that supports Gethli's reasoning pass, future flag reliability scoring, and cross-check corroboration. Key fields include: event identity (`event_id`, `timestamp`, `session_id`, `interaction_id`); trigger metadata (`trigger_key`, `trigger_registry_version`, `flag_schema_version`, `agent_profile_version`, `emitter`); the raw agent flag payload stored as an untrusted claim alongside its parse status; the capture bundle references; a context fingerprint (SHA-256 hash of capture context, never the content itself); and two assessment fields — `provenance_note` (populated by Gothi) and `gethli_assessment` (null until Gethli's second pass, then filled with classification, severity, finding determination, and corroboration evidence).

The separation of `provenance_note` from corroboration is architecturally significant. Provenance — this flag came from this agent at this time — is known immediately. Corroboration — this cooperative claim is supported by independent reactive evidence — requires session history to compute meaningfully and is deferred until the reliability scoring phase.

§ 6

Agent Delivery Formats and Compute Economics

The architecture supports two delivery formats for the audit contract: compliant agent structures, in which the audit contract is woven natively into the agent persona from the ground up, and add-in blocks, which inject the contract paragraph into existing agent profiles. The distinction has significant implications for compliance quality and compute economics.

Compliant agent structures achieve higher flag validity because flagging behavior is native to the agent's identity rather than an adaptation of a base persona that may compete with the contract under long context. The Wiz agent described in the implementation section is an example of a compliant structure: the audit contract appears as the first substantive section after the agent's role and mission, uses blocking requirement language identical to the agent's existing SOP compliance mechanism, and treats the contract as load-bearing identity rather than an injected rule.

The compute economics of cooperative capture are favorable relative to reactive-only systems in a specific and important way. Without pre-flagging, an audit reasoner receives raw captures and must perform the full reconstruction stack on every pass: event detection, categorization, severity assessment, and provenance reconstruction across the entire session context. With pre-flagging, the reasoner receives pre-labeled, pre-categorized events with bounded context. The reconstruction work is done at generation time — which costs nothing additional since the agent was generating anyway — and the reasoner's task

shifts from reconstruction to verification. This is a materially narrower reasoning task that a smaller model can perform at higher accuracy than a larger model attempting reconstruction from raw captures.

This suggests a tiered Gethli model strategy: lightweight local models for routine flag verification on well-labeled events, with escalation to more capable models for complex provenance reconstructions and multi-hop contradiction detection. The cooperative capture layer is what makes that tiering tractable — without pre-labeled events, the system cannot reliably identify which events require the heavier model until after it has already spent the compute determining that.

§ 7

Implementation Evidence

The architecture described in this paper was implemented across thirteen phases (EF-1 through EF-13) in a TypeScript VS Code extension, with EF-13 (flag reliability scoring) explicitly deferred pending sufficient session history. The implementation plan was derived directly from the architecture specification by an IDE agent (Wiz) operating under the cooperative audit contract described above — the system was used to build itself.

A post-sprint audit, conducted by Wiz following the Code Review SOP, produced four findings across 353 passing tests, 0 type errors, and 0 lint errors. The architecture invariant gate confirmed that all nine new engine modules were free of `vscode.* imports`, maintaining the abstraction boundary that keeps future host connectors (Xcode, CI integration) viable without porting the engine. The four findings were runtime wiring gaps — `AuditEventRecorder` instantiated but never wired to `EventManager`, `submitAgentFlag` command missing from `package.json` — rather than logic errors, demonstrating that the audit trail surfaces the class of silent failure most likely to escape conventional quality gates.

The single cooperative audit marker emitted during the review session — `[[AUDIT:GROUNDING_GAP | context: claim that EF-5 and EF-7 wiring gaps are runtime-silent — confirmed by grep]]` — demonstrates the cooperative mechanism working as intended: the agent flagged a factual claim as requiring grounding, then immediately grounded it through terminal verification. The marker and the evidence are co-located in the session record. The audit trail is replayable.

The test coverage exceeded the planned target: 76 new tests delivered against a plan of approximately 65, with the single missing test file (`auditEventAssessor.test.js`) mapping directly to the missing EF-11 implementation. Coverage gaps and implementation gaps pointed at the same thing. This correlation — absent in conventional testing-only approaches — is a property of the audit architecture: the trail has enough structure to make gaps visible rather than silent.

§ 8

Toward Engineering-Grade Software Development

The cooperative audit architecture represents a structural attempt to close the gap between software development and engineering discipline. That gap, as argued throughout this paper, is not primarily a gap in effort, talent, or tooling for artifact-level quality — it is a gap in the instrumentation of the reasoning layer. Engineering disciplines do not achieve rigor by asking practitioners to try harder; they achieve it by making certain failures visible before they compound, through the structure of the work itself.

The audit architecture operates at the layer where the gap has always been widest. An intent-indexed event record, generated contemporaneously with the code it describes, is structurally different from a retrospective review of a completed artifact. The design record and the artifact coexist. The decision trail is part of the deliverable. This is how aerospace software is developed under DO-178C; it is how pharmaceutical processes are documented under Good Laboratory Practice. The Gothi/Gethli system brings the same structural property to AI-assisted software development without requiring formal specification or domain-specific truth criteria.

Several important qualifications limit the scope of this claim. The architecture requires sustained agent compliance to deliver its cooperative capture benefits — compliance that can degrade under long context, complex generation pressure, or add-in block delivery formats. The reactive fallback layer ensures the floor does not drop, but the ceiling with cooperative capture in good compliance is meaningfully higher. Additionally, the audit trail only compounds into genuine discipline if it is treated as a first-class artifact: reviewed, queried, and used to inform future sessions. A system whose records are generated but never consulted produces data without learning.

The flag reliability scoring system — deferred pending sufficient session history — represents the mechanism by which the architecture becomes empirically self-improving. Once the database contains enough session records to establish meaningful correlation patterns, Gethli can compute per-agent compliance profiles: corroboration rates (cooperative flags confirmed by reactive evidence), omission rates (reactive events with no preceding cooperative flag), and false positive rates (cooperative flags with no corroborating evidence). These profiles are, in the language of empirical software engineering, evidence-based assessments of agent behavior.¹⁶ They transform practitioner intuition about AI tool quality into queryable, time-stamped data.

The broader implication for software engineering practice is this: the profession has had almost no instrumentation at the reasoning layer. It has been possible to measure test coverage, deployment frequency, defect rates, and cycle times — but not the quality of the decision-making that produced the code those metrics describe. Cooperative audit begins to instrument that layer. Whether it compounds into something that genuinely raises the engineering floor depends on how consistently the record is used — but for the first time, the record exists.

Conclusion

This paper has argued that the gap between software development and engineering rigor is fundamentally a gap in provenance: the reasoning that produces code has been structurally invisible, and the emergence of AI-assisted development has widened that invisibility at velocity. The cooperative audit architecture proposed here addresses this gap by treating working AI agents as cooperative participants in a declarative audit framework, generating intent-indexed event records at the moment of production rather than reconstructing them after the fact.

The system does not claim to validate code. It claims to audit it — to record what changed, where it came from, what contradicts what, and how reliably the agents contributing to the record are doing their part. This is the audit promise that mature engineering disciplines have long required and that software development has long deferred. The architecture described here makes it structurally available, for the first time, at the layer that matters most: the reasoning that produces the artifact.

Bibliography

1. RTCA. *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. Washington, DC: RTCA, 2011. See also: NASA Langley Research Center. "Certification of Safety-Critical Software Under DO-178C." NASA Technical Reports Server, 2012.
<https://ntrs.nasa.gov/api/citations/20120016835/downloads/20120016835.pdf>.
2. Begel, Andrew, and Thomas Zimmermann. "Analyze This! 145 Questions for Data Scientists in Software Engineering." *Proceedings of the 36th International Conference on Software Engineering*. New York: ACM, 2014. See discussion in: Devanbu, Prem, Thomas Zimmermann, and Christian Bird. "Belief and Evidence in Empirical Software Engineering." *Proceedings of ICSE 2016*. New York: ACM, 2016.
3. Spanoudakis, George, and Andrea Zisman. "A Survey of Traceability in Requirements Engineering and Model-Driven Development." *Software and Systems Modeling* 9, no. 4 (2009): 415–452.
<https://doi.org/10.1007/s10270-009-0145-0>.
4. Cleland-Huang, Jane, Orlena Gotel, and Andrea Zisman, eds. *Software and Systems Traceability*. London: Springer, 2012. See also: Mäder, Patrick, and Alexander Egyed. "Do Developers Benefit from Requirements Traceability When Evolving and Maintaining a Software System?" *Empirical Software Engineering* 20, no. 2 (2015): 413–441. Quotation from: Cleland-Huang, Jane, et al. "Software Traceability: Trends and Future Directions." *Future of Software Engineering Proceedings*. New York: ACM, 2014. <https://dl.acm.org/doi/10.1145/2593882.2593891>.
5. Munroe, Steve, Simon Miles, Luc Moreau, and Javier Vázquez-Salceda. "PrIMe: A Software Engineering Methodology for Developing Provenance-Aware Applications." *Proceedings of the 6th International Workshop on Software Engineering and Middleware*. New York: ACM, 2006. See also: Herschel, Melanie, Ralf Diestelkämper, and Houssem Ben Lahmar. "A Survey on Provenance: What for? What form? What from?" *VLDB Journal* 26, no. 6 (2017): 881–906. Quotation from ResearchGate summary of: Groth, Paul, et al. "Provenance of Software

- Development Processes." *Provenance and Annotation of Data and Processes*. Berlin: Springer, 2010.
<https://www.researchgate.net/publication/225005835>.
6. Nadella, Satya, and Sundar Pichai, as cited in: Hao, Karen. "AI Coding Is Now Everywhere. But Not Everyone Is Convinced." *MIT Technology Review*, December 15, 2025.
<https://www.technologyreview.com/2025/12/15/1128352/rise-of-ai-coding-developers-2026/>.
 7. Stack Overflow. *Developer Survey 2024*. As cited in: "AI-Driven Automation of Code Review Processes." *UL Open Access*, 2025. https://ulopenaccess.com/papers/ULIRS_V02I02/ULIRS20250202_006.pdf.
 8. GitClear. *AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones*. GitClear Research, 2025.
https://www.gitclear.com/ai_assistant_code_quality_2025_research.
 9. Qodo. *State of AI Code Quality 2025*. Qodo Research, June 2025. <https://www.qodo.ai/reports/state-of-ai-code-quality/>.
 10. Hao, Karen. "AI Coding Is Now Everywhere. But Not Everyone Is Convinced." *MIT Technology Review*, December 15, 2025. <https://www.technologyreview.com/2025/12/15/1128352/rise-of-ai-coding-developers-2026/>.
 11. Osmani, Addy. "My LLM Coding Workflow Going Into 2026." *AddyOsmani.com*, January 4, 2026.
<https://addyosmani.com/blog/ai-coding-workflow/>.
 12. Vance, Ashlee. "Empirical Software Engineering." *American Scientist* 106, no. 1 (January–February 2018).
<https://www.americanscientist.org/article/empirical-software-engineering>.
 13. RTCA. *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. Washington, DC: RTCA, 2011. Section 6 objectives summarized in: NASA Langley Research Center, 2012 (see note 1). See also: arc42 Quality Model. "DO-178C." <https://quality.arc42.org/standards/do-178c>.
 14. NASA. *NASA Software Engineering Handbook, NPR 7150.2C*. Washington, DC: NASA, 2019. SWE-059: Bidirectional Traceability Between Software Requirements and Software Design.
<https://swehb.nasa.gov/display/7150/SWE-059>. Quotation from: SWE-047: Traceability Data.
<https://swehb.nasa.gov/display/SWEHBVB/SWE-047>.
 15. Young, Greg. "CQRS and Event Sourcing." CodeBetter.com, February 2010.
<http://codebetter.com/gregyoung/2010/02/13/cqrs-and-event-sourcing/>. See also transcript of: Young, Greg. "CQRS and Event Sourcing." Talk at Code on the Beach, 2014. Transcript available at:
<https://www.kurrent.io/blog/transcript-of-greg-youngs-talk-at-code-on-the-beach-2014-cqrs-and-event-sourcing>.
 Summary in: "The Basics of Event Sourcing and Some CQRS." *InfoQ*, September 30, 2014.
<https://www.infoq.com/news/2014/09/greg-young-event-sourcing/>.
 16. Kitchenham, Barbara, and Stuart Charters. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Keele University Technical Report EBSE-2007-01, 2007. See also: Dybå, Tore, and Torgeir Dingsøy. "Empirical Studies of Agile Software Development: A Systematic Review." *Information and Software Technology* 50, nos. 9–10 (2008): 833–859.